SZABIST, Islamabad Campus

MezFlow

A Real-Time Restaurant Dining and Kitchen Operations Management System

Final Year Project Proposal by

Sheikh Muhammad Khizar (2380227)
Talha Mumtaz (2380700)
Syed Sail Haider Gardezi (2380245)

Supervised by: Muhammad Farooq

Revision History

Compiled By	Checked By	Date	Reason for Change	Version
Sheikh Muhammad Khizar	M. Farooq	21th Sep 2026	Initial Version	1.0
Syed Sail Haider	M. Farooq	23th Sep 2026	Introduction Revised	1.2
Talha Mumtaz	M. Farooq	25th Sep 2026	References Revised	1.3

Project Description

MezFlow is a web and mobile platform that connects the customer, kitchen, billing and management workflows of a restaurant through a single real-time system. It combines a zero-install, QR-activated Progressive Web App through which diners at the same table share one live order, a single-screen Kitchen Display System (KDS) that replaces paper tickets with a synchronized digital order queue, a management console for menu, table and staff configuration, and a grounded AI feature that explains dishes using only restaurant-provided information. A real-time layer built on WebSockets keeps the customer app, the KDS and the management console synchronized as an order moves from New to Confirmed, Preparing, Ready and Dispatched. The platform also tracks the physical table separately from the digital session, so a table only becomes available again once staff confirm it has been cleaned — payment alone does not free it. The pilot deployment targets small and medium dine-in restaurants, cafés and food courts that currently rely on printed menus, verbal ordering and manual kitchen coordination. The design is restaurant-agnostic, so additional locations, languages and payment methods can be added later without a redesign.

1 Introduction

Restaurant service depends on tight coordination between customers, waiters, kitchen staff, cashiers and managers. In most small and medium restaurants, this coordination still runs on printed menus, verbal communication, handwritten or manually entered tickets, and separate tools for ordering, billing and menu management. Digital point-of-sale (POS) systems and QR-ordering apps have automated parts of this process, but in the small-restaurant segment they typically stop at showing a menu or printing a kitchen ticket — collaborative ordering, live kitchen visibility, delay detection and the distinction between a paid bill and a physically clean table are rarely handled together.

This creates avoidable delays and errors. Customers struggle with unfamiliar dishes and wait for a waiter to take or modify an order; kitchen staff have limited visibility into the full order queue during a rush; and managers cannot easily see which orders are late or which tables are actually free. During peak hours, when ten or more tables may order within minutes, a purely manual or lightly digitized workflow leaves staff to track table numbers, items, modifiers and timing by memory or on paper.

MezFlow is proposed to close this gap with an integrated, real-time platform: customers scan a table QR code to join a shared order with no app to install; the kitchen sees every order on one synchronized screen; the backend tracks preparation time and flags delays automatically; and management configures the menu, tables and staff from a single console. The rest of this proposal reviews existing restaurant technology (Section 2), states the problem (Section 3), defines the aim and objectives (Section 4), describes the modules and scope (Section 5), explains the development methodology and architecture (Section 6), lists the tools and technologies (Section 7) and presents the work plan (Section 8).

Core workflow: Customer → Table Session → Order → Kitchen → Preparation → Serving → Bill → Payment → Table Cleanup → Available

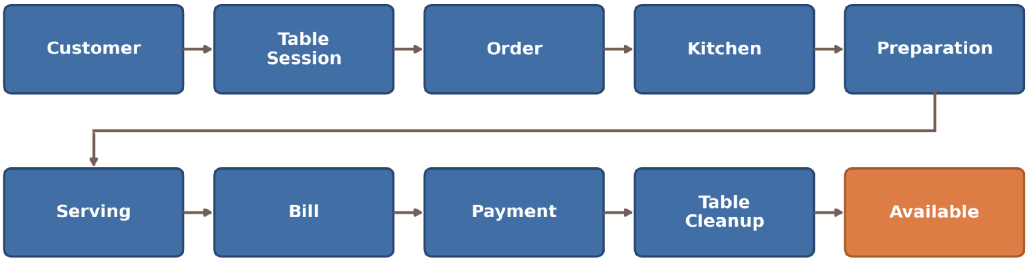


Figure 1: MezFlow end-to-end workflow, from customer arrival to table availability.

2 Application/Literature Review

This section reviews three widely used restaurant-technology products — a proprietary POS-and-KDS system, a free-tier small-business POS platform, and a dedicated QR table-ordering product — and identifies the gap that MezFlow is designed to close.

Toast POS & KDS

Toast is a widely deployed, US-based restaurant point-of-sale platform that bundles ordering, payment processing and a proprietary Kitchen Display System on dedicated hardware terminals [Ref: 4]. It is aimed at full-service and quick-service restaurants that can commit to its hardware and contract model.

- Integrated point-of-sale and kitchen display system on proprietary hardware.
- Online ordering and third-party delivery integrations.
- Menu and pricing updates synchronized across terminals.
- Sales, labor and inventory reporting dashboard.
- Optional customer-facing handheld ordering devices.

The limitations of the platform are listed below.

- Requires proprietary POS hardware and multi-year contracts, which raises cost for small restaurants.
- No collaborative, QR-based multi-diner ordering with a shared table session.
- The KDS only reflects what is entered at the POS; no IoT, sensor or delay-prediction input.
- No customer-facing AI or dish-explanation features.
- Payment closure and physical table availability are not modeled as separate states.

Square for Restaurants

Square for Restaurants is a free-tier POS application that runs on iPad or Square hardware [Ref: 5] and is popular with small and medium food businesses because of its low upfront cost and simple setup.

- Free-tier POS software with an optional Kitchen Display System app.
- Auto-generated online ordering website.
- Menu management synchronized across in-store and online channels.
- Bundled card payment processing.
- Basic sales and item-performance reporting.

The limitations of the platform are listed below.

- QR ordering is menu-only; there is no shared cart that multiple diners at one table can add to live.

- The KDS app is tied to Square's own ecosystem rather than being an independent, low-cost display.
- No explicit table lifecycle — tables are not tracked through available/occupied/cleanup states.
- No AI-assisted dish explanation or grounded content generation.
- Kitchen category routing is limited to basic printer/station assignment.

mr yum (QR Table Ordering)

mr yum is a QR-code ordering and payment platform used by hospitality venues in Australia, the UK and the US [Ref: 6]. Customers scan a code at their table, browse the menu in a browser, and order and pay without installing an app.

- Zero-install, browser-based QR table ordering.
- Direct-to-kitchen ticket printing on order submission.
- Item modifiers and customization options.
- Integrated at-table payment.
- Menu-item performance analytics for venues.

The limitations of the platform are listed below.

- Built around one order per scan; limited support for several diners building one live shared cart.
- Relies on ticket printers rather than a single-screen KDS with category filters and delay timers.
- No ground-level sensor or environmental data feeding kitchen or safety decisions.
- No grounded AI dish explanation — menu information is static text supplied by the venue.
- Table cleanup/availability after payment is left entirely to staff with no system state.

Table 1 compares the systems reviewed against MezFlow, based on their publicly documented features.

Table 1: Applications Comparison

Feature	Toast POS & KDS	Square for Restaurants	mr yum	Proposed (MezFlow)
Collaborative multi-diner table session	✗	✗	✗	✓
Zero-install QR/PWA customer access	✗	✗	✓	✓
Software-only single-screen KDS	✗	✗	✗	✓
Real-time WebSocket order synchronization	✗	✗	✗	✓
Preparation-time tracking & delay detection	✗	✗	✗	✓
Separate digital / physical table lifecycle	✗	✗	✗	✓
Grounded AI dish explanation	✗	✗	✗	✓
No proprietary hardware or long-term contract	✗	✓	✓	✓

3 Problem Statement

Many small and medium restaurants still run on partially manual workflows for ordering, kitchen coordination, menu management and table turnover. Customers often cannot get a clear explanation of an

unfamiliar dish without a waiter, and modifiers communicated verbally (spice level, removed ingredients, portion changes) are easy to mishear or forget between the table and the kitchen.

The core problem is not the absence of digital tools but the absence of one connected, real-time workflow. Four specific gaps stand out:

1. **Ordering is fragmented across diners.** When several people share a table, there is usually no single shared order — either one person orders for everyone verbally, or each diner must flag a waiter separately.
2. **Kitchen visibility is limited.** Paper tickets or a printer queue give the kitchen no organized, filterable view of every active order, and no automatic way to flag a delayed dish.
3. **Digital tools stop at the menu or the ticket.** Existing POS and QR-ordering products (Section 2) rarely connect ordering, kitchen operations, billing and table-state tracking in one system, especially for small restaurants that cannot afford proprietary hardware.
4. **Table turnover is assumed, not tracked.** Most systems treat a paid bill as the end of the workflow, so nothing distinguishes a table that is paid-but-still-seated from one that is actually clean and ready for the next guests.

The users who need a solution, and their stake in it, are:

- **Restaurant owners/managers,** who need lower order-taking overhead, fewer mistakes, and visibility into which orders and tables need attention.
- **Kitchen staff,** who need one organized, prioritized queue instead of scattered tickets during a rush.
- **Waiters and service staff,** who need fewer repetitive trips to take orders or answer menu questions.
- **Cashiers,** who need an accurate, itemized bill tied to a verified session rather than a hand-tallied one.
- **Customers,** who want to order at their own pace, understand what they are ordering, and track when their food will arrive.

4 Project Aim and Objectives

Aim. To design, build and test an integrated, real-time platform that connects customer ordering, kitchen operations, billing and restaurant management for small and medium dine-in restaurants, using a zero-install web app, a single-screen kitchen display, and a grounded AI dish-explanation feature.

Objectives:

1. Build a zero-install, QR-activated Progressive Web App through which customers browse the menu and order, targeting a first-load time under 3 seconds on a typical mobile connection.
2. Implement collaborative table sessions in which multiple diners scanning the same QR code share one live order, with the backend still recording which diner added each item.
3. Build a real-time layer (WebSockets) that propagates order, kitchen and table-status changes to every connected client within 2 seconds under normal network conditions.
4. Develop a single-screen Kitchen Display System that displays every active order as an independent card and supports category and status filtering, targeting correct, non-interfering display of at least 10 simultaneous table orders submitted within a 2-minute window.
5. Implement preparation-time tracking that compares elapsed time against a configurable expected time per item and automatically flags delayed orders on the KDS.

6. Implement an order lifecycle (New → Confirmed → Preparing → Ready → Dispatched) and a table lifecycle (Available → Occupied → Awaiting Cleanup → Available) as independent, auditable state machines.
7. Build a digital billing workflow in which a customer requests the bill, an authorized staff member processes payment, and the session is closed without assuming the table is physically free.
8. Provide a grounded "Explain This Dish" AI feature that generates a natural-language explanation strictly from restaurant-provided structured data, without inventing facts not supplied by the restaurant.
9. Build a management console for menu, table, staff and pricing configuration, protected by role-based access control (customer, kitchen staff, service staff, cashier, manager).
10. Evaluate the platform through load testing (at least 50 concurrent WebSocket connections without state corruption) and a usability trial with at least 10 participants, targeting a System Usability Scale (SUS) score of at least 70.

Value proposition. Unlike Toast, MezFlow needs no proprietary hardware or long-term contract, which keeps it accessible to small and independent restaurants. Unlike Square for Restaurants, it supports a genuinely collaborative shared table cart and models the table's physical lifecycle explicitly rather than assuming payment means the table is free. Unlike mr yum, its Kitchen Display System is a first-class, software-only module with category filtering and automatic delay detection rather than a ticket printer, and it adds a grounded AI dish-explanation feature that none of the three reviewed systems offer. Because it is built entirely on open-source frameworks and standard web protocols, it can be self-hosted at low cost by an independent restaurant or a small chain.

Final output and packaging. The project will deliver: a customer Progressive Web App (React/Next.js) requiring no installation; a Kitchen Display System web application; a management console web application; a backend REST and WebSocket API (Python/FastAPI) with a PostgreSQL database; and documentation comprising an SRS, a design document (architecture, ER diagram, API specification), a test report, and user and deployment manuals. No proprietary hardware is required — the KDS runs on any browser-capable tablet, laptop or low-cost computer already available to the restaurant.

5 Scope and Significance

5.1 Project Scope

The pilot deployment targets independent, small and medium dine-in restaurants, cafés and food-court outlets with between roughly 5 and 30 tables, where staff currently rely on verbal or paper-based ordering. The platform consists of the following twelve modules.

Module 1: Identity & Access Control

- Sign-up and login by phone number or e-mail with OTP verification and JWT-based sessions.
- Role-based access: Customer, Kitchen Staff, Service Staff, Cashier, Manager.
- Profiles with preferred language (Urdu/English) and notification preferences.
- Audit log of every staff action (menu changes, order overrides, bill approvals).

Module 2: Digital Menu & QR Table Access

- QR code per table opens the menu directly in the browser — no app installation.
- Menu categories, item descriptions, images, pricing and live availability.
- Structured modifiers: spice level, quantity, size, add-ons, ingredient removal, notes.
- Automatic reflection of price or availability changes made by the manager.

Module 3: Collaborative Table Ordering

- Multiple diners scanning the same table QR code join one shared table session.
- Shared cart with per-item attribution to the diner who added it.
- Real-time cart updates visible to every diner at the table.
- Order submission that creates a single order per table for the kitchen.

Module 4: Kitchen Display System (KDS)

- Single-screen application showing every active order as an independent card.
- Card details: order number, table number, items, modifiers, elapsed time, status.
- Filters by status (New, Preparing, Ready, Delayed) and by kitchen category (Grill, Fry, Hot Kitchen, Beverage, etc.).
- Runs on any tablet, laptop or low-cost computer — no dedicated hardware required.

Module 5: Order Lifecycle & Real-Time Synchronization

- Order state machine: New → Confirmed → Preparing → Ready → Dispatched.
- WebSocket events propagate every state change to the customer app, KDS and management console.
- Customers see live order status without refreshing the page.
- Independent handling of simultaneous orders from different tables with no cross-order interference.

Module 6: Preparation-Time Tracking & Delay Detection

- Configurable expected preparation time per menu item.
- Automatic elapsed-time calculation from the moment preparation starts.
- Delayed-order flag on the KDS when elapsed time exceeds the expected time.
- Kitchen category routing so a customer-facing category (e.g. “Main Course”) can map to an internal kitchen station (e.g. “Hot Kitchen”).

Module 7: AI-Assisted Dish Explanation

- “Explain This Dish” feature generating a natural-language description from restaurant-supplied fields (ingredients, flavor, texture, heat level, portion size).
- Explanations are grounded strictly in restaurant-provided data — no invented facts.
- Support for Urdu and English output.
- Manager control over which fields are supplied per dish.

Module 8: Billing & Payment Workflow

- Customer-initiated bill request from the app.
- Cashier review and payment processing (cash or card).
- Server-side price validation — the backend never trusts a price sent from the customer device.
- Session closure recorded with a full audit trail.

Module 9: Table Lifecycle Management

- Table state machine: Available → Occupied → Awaiting Cleanup → Available.
- Payment closes the digital session but does not automatically free the table.
- Staff-confirmed cleanup step before a table returns to Available.
- Live table-status view for service staff and managers.

Module 10: Management Console

- Menu management: create/edit items, prices, availability, categories, preparation times.
- Table management: create tables, generate QR codes, monitor active sessions.
- Staff management: role assignment and permissions.
- Restaurant-level configuration (taxes, service charges, operating hours).

Module 11: Service Requests & Notifications

- In-app request for a waiter, water refill, or other assistance.
- Push notifications for order status, bill readiness, and service request acknowledgement.
- Notification queue visible to service staff, sorted by table and wait time.

Module 12: Analytics & Operational Records

- Dashboard of orders, average preparation time, and delayed-order rate by date and category.
- Full operational audit trail (order creation, state changes, payments, table status changes).
- Exportable reports for management review.

Out of Scope

To keep the project achievable within the FYP timeline, the following are excluded:

- Food delivery/rider marketplace integration.
- Full accounting/ERP or payroll systems.
- Inventory and supply-chain management beyond basic availability toggles.
- Custom POS hardware, biometric or facial-recognition identification.
- Native mobile applications (the customer experience is delivered as a Progressive Web App).
- Automated physical table-cleanliness detection — cleanup is confirmed by staff, not sensed.

5.2 Project Significance

- Replaces fragmented, verbal ordering with one collaborative, shared table session.
- Gives the kitchen a single, organized, filterable view of every active order instead of scattered paper tickets.
- Separates digital session closure from physical table readiness, preventing a common source of confusion during turnover.
- Requires no proprietary hardware, making it affordable for independent and small-chain restaurants.
- Automatic delay detection helps management catch bottlenecks in the kitchen before customers complain.
- Grounded AI dish explanations reduce dependence on waiter availability for menu questions.
- Modular, real-time architecture that can extend to more locations, languages or payment methods without a redesign.

6 Project Development Methodology

The project will follow the Agile Scrum methodology with two-week sprints. The twelve modules are broken into user stories in a product backlog, and each sprint ends with a working increment reviewed with the supervisor. Agile suits this project because two areas carry genuine uncertainty — how the collaborative table-session model behaves under concurrent multi-diner use, and how kitchen staff actually want to filter

and act on the KDS queue during a rush — and an iterative approach lets the team adjust based on early feedback. Requirements will be gathered through a review of existing POS and QR-ordering products (Section 2), informal interviews with restaurant staff, and usability walkthroughs with prospective diners.

Work is split across the two FYP semesters. FYP-I builds the data foundation and a working prototype of the ordering path: requirements and design, Modules 1–4 and Module 8's billing groundwork. FYP-II adds real-time synchronization, kitchen operations, AI and analytics (Modules 5–7 and 9–12), followed by integration, load testing, a usability trial and documentation. Figure 2 shows the system-level architecture and Table 2 lists the deliverable of each module.

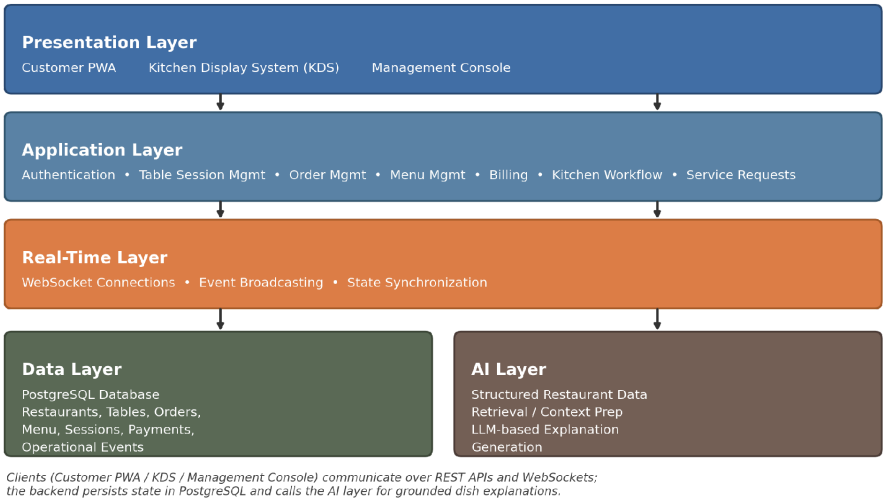


Figure 2: Layered system architecture of MezFlow.

Table 2: Module-wise Objectives and Deliverables

No.	Module	Key Deliverable	Phase
M1	Identity & Access Control	Auth API with five roles, OTP login, audit log	FYP-I
M2	Digital Menu & QR Access	QR-activated menu with live pricing/availability	FYP-I
M3	Collaborative Table Ordering	Shared cart with per-diner item attribution	FYP-I
M4	Kitchen Display System	Single-screen KDS with status/category filters	FYP-I
M5	Order Lifecycle & Real-Time Sync	WebSocket event layer synchronizing all clients	FYP-II
M6	Preparation-Time & Delay Tracking	Configurable timers and automatic delay flags	FYP-II
M7	AI Dish Explanation	Grounded explanation generator (Urdu/English)	FYP-II
M8	Billing & Payment Workflow	Bill request → payment → session closure flow	FYP-I/II
M9	Table Lifecycle Management	Available/Occupied/Cleanup state machine	FYP-II
M10	Management Console	Menu, table and staff administration UI	FYP-II
M11	Service Requests & Notifications	In-app requests and push notification queue	FYP-II
M12	Analytics & Operational Records	Dashboard, audit trail, exportable reports	FYP-II

Order and Table Lifecycle Design

The order lifecycle and the table lifecycle are modeled as independent state machines so that payment — which closes the digital session — never assumes the physical table is already clean and free for the next guests. Figure 3 shows both lifecycles.

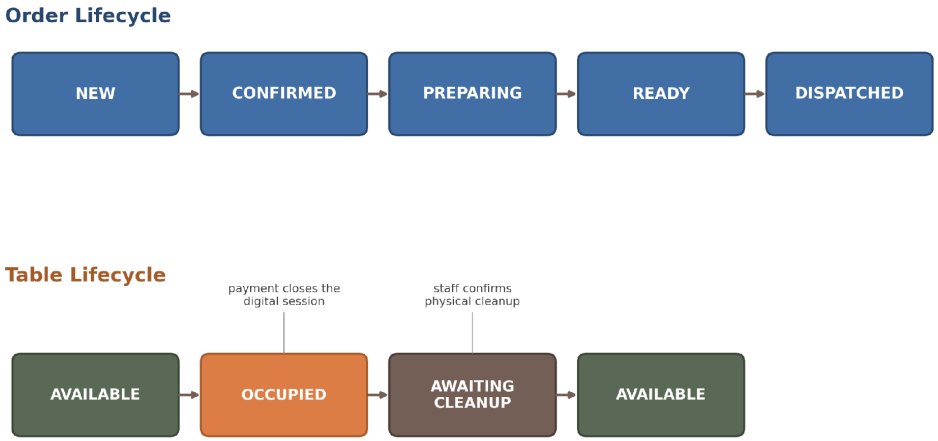


Figure 3: Order lifecycle (top) and table lifecycle (bottom).

Database Design

The database is organized around a restaurant, its tables, customer sessions and diners, the menu catalog, orders and order items, payments, and an operational-event log used for the audit trail and analytics. Figure 4 shows the entity-relationship design.

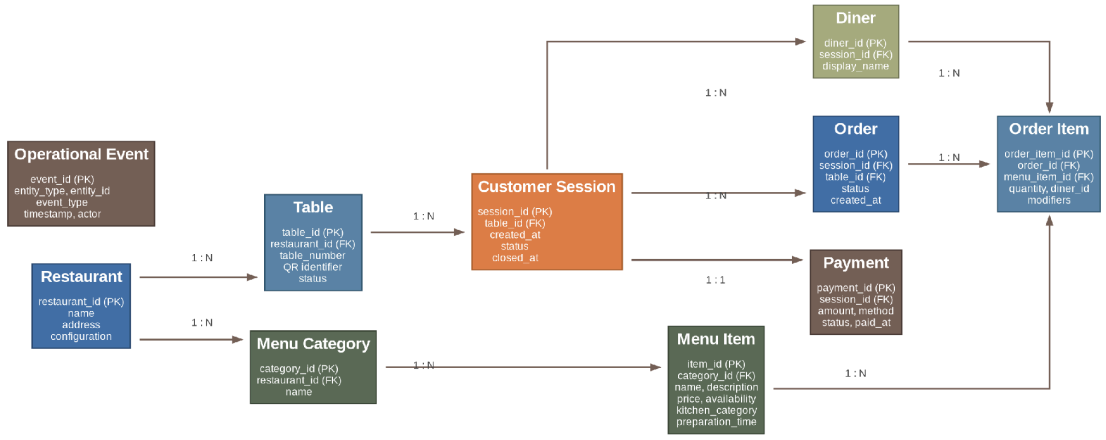


Figure 4: Entity-relationship diagram of the MezFlow database schema.

Evaluation Plan

The platform will be evaluated on:

- **Real-time performance:** order and status changes reach all connected clients within 2 seconds under normal network load.
- **Concurrency:** at least 50 simultaneous WebSocket connections and 10 simultaneous table orders handled without state corruption or cross-order interference.

- **Correctness:** automated unit, API and integration tests covering the order lifecycle, table lifecycle, and price-validation logic.
- **Usability:** a System Usability Scale (SUS) score of at least 70 from 10 or more participants, including non-technical users.
- **AI grounding:** manual review confirming dish explanations do not introduce facts absent from the restaurant-provided data.

Risks and Mitigation

Table 3: Key Risks and Mitigation

Risk	Mitigation
Concurrent edits to a shared table cart causing lost updates	Server-authoritative cart state; WebSocket broadcast of every change; optimistic UI reconciled against server state
WebSocket connection drops on unstable restaurant Wi-Fi	Automatic reconnect with state resync; REST fallback for critical reads
Kitchen staff resistance to a new digital workflow	Simple, filter-driven KDS UI; on-site training during the usability trial; staff feedback folded into sprint backlog
AI dish explanations drifting from restaurant-provided facts	Strict prompt grounding to supplied fields only; manual review step before publishing a dish description
Scope creep across twelve modules within the FYP timeline	Fixed two-week sprints; MVP-first module order; out-of-scope list enforced by the supervisor review

7 Tools and Technologies

Table 4 lists the tools and technologies that will be used to implement the project. All are free, open-source or available on a free tier.

Table 4: Tools and Technologies

Layer	Tools / Technologies	Purpose
Frontend	React / Next.js, TypeScript, Tailwind CSS	Customer PWA, KDS and management console UI
Backend API	Python, FastAPI	REST and WebSocket APIs, business logic
Real-Time Communication	WebSockets (RFC 6455)	Live order, kitchen and table-state synchronization
Database	PostgreSQL	Restaurants, tables, sessions, menu, orders, payments, events
AI	LLM integrated with structured, restaurant-owned context	Grounded dish explanations
DevOps	Git/GitHub, GitHub Actions, Docker, Nginx, cloud VPS	CI and deployment
Design & Docs	Figma, draw.io, Overleaf (LaTeX)/Word	UI design, diagrams, documentation
Testing	pytest, Postman, Locust	Unit, API and load testing

8 Work Plan

8.1 Team Structure

Table 5 defines the role of each team member.

Table 5: Team Structure

Sr. No.	Team Members	Role
1	Talha Mumtaz	Backend Developer —D atabase design, real-time services
2	Syed Sail Haider	Frontend Developer — Kitchen Display System & Customer PWA and collaborative ordering
3	Sheikh Muhammad Khizar	Full-Stack Developer — Management console, AI module, QA, REST APIs

8.2 Work Distribution

Table 6 shows the division of work. Each member owns three modules; shared activities are done by the whole team.

Table 6: Work Distribution

Sr. No.	Team Member	Work Assignment
1	Sheikh Muhammad Khizar	M1 Identity & Access, M5 Order Lifecycle & Real-Time Sync, M8 Billing & Payment; backend architecture
2	Syed Sail Haider	M2 Digital Menu & QR Access, M3 Collaborative Table Ordering, M9 Table Lifecycle; UI/UX for the customer app
3	Talha Mumtaz	M4 Kitchen Display System, M6 Preparation-Time Tracking, M11 Service Requests & Notifications; real-time testing
4	Sheikh Muhammad Khizar	M7 AI Dish Explanation, M10 Management Console, M12 Analytics & Operational Records; QA and documentation
—	All members	Requirements, SRS and design, integration, backtesting, usability trial, documentation

8.3 Gantt Chart

Figure 5 gives a visual representation of the overall development process in terms of tasks, timeline and dependencies over the 16-week implementation window.

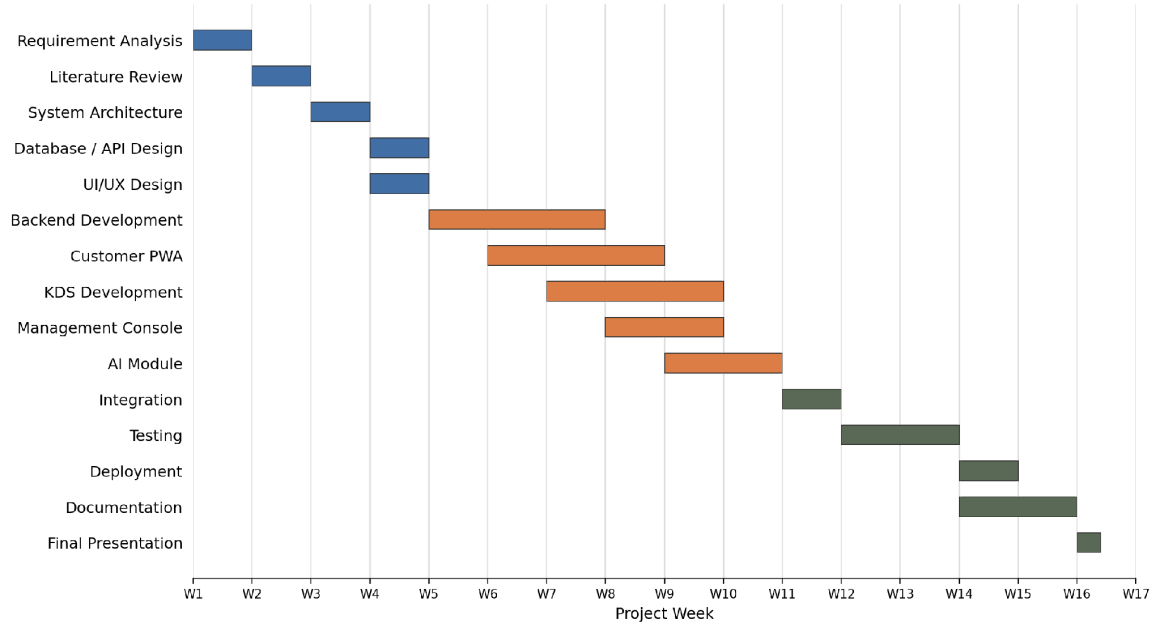


Figure 5: Gantt Chart

References

1. IETF. RFC 6455 — The WebSocket Protocol [Online]. Available: https://www.rfc-editor.org/rfc/rfc6455. Last Accessed on September 25, 2026.
2. MDN Web Docs. Progressive Web Apps — Best Practices [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps. Last Accessed on September 25, 2026.
3. MDN Web Docs. Offline and Background Operation [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Offline_and_background_operation. Last Accessed on September 25, 2026.
4. Toast, Inc. Toast POS and Kitchen Display System [Online]. Available: https://pos.toasttab.com. Last Accessed on September 25, 2026.
5. Block, Inc. Square for Restaurants [Online]. Available: https://squareup.com/us/en/point-of-sale/restaurants. Last Accessed on September 25, 2026.
6. Mr Yum Pty Ltd. Mr Yum — QR Ordering and Payments [Online]. Available: https://mryum.com. Last Accessed on September 25, 2026.
7. J. Brooke, “SUS: A quick and dirty usability scale,” in Usability Evaluation in Industry, P. W. Jordan et al., Eds. London, UK: Taylor & Francis, 1996, pp. 189–194.